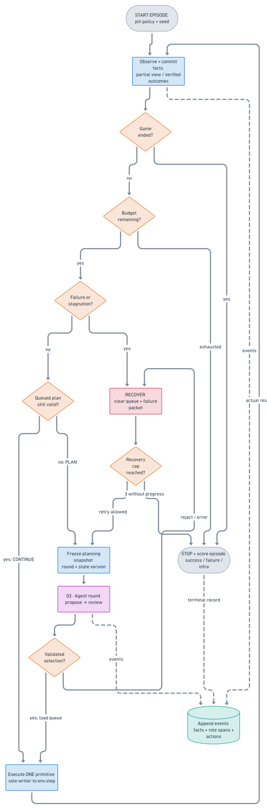
GAME AGENT LAB

Runtime, evaluation and controlled self-improvement

Read 01 → 02 for gameplay. Read 03 → 04 for improvement and evidence.
Blue = deterministic code · Pink = LLM role · Orange = routing gate · Mint = evidence/evals · Red = recovery/rejection.
Three online roles; two concurrent requests. One offline Failure Analyst. One game writer. Model weights stay fixed.

01 · EPISODE CONTROL LOOP

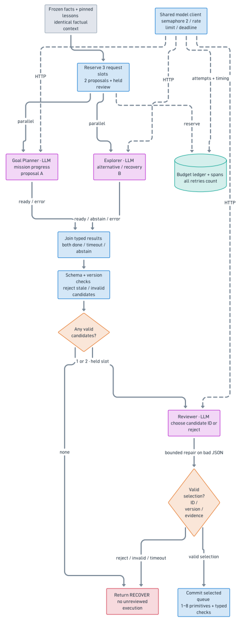
Continue, plan, recover or stop. Policy is fixed for the whole episode.



Routing contract
Terminal → STOP. Action/time cap → STOP.
Failed action / repeated failure / 8 actions without progress → RECOVER.
Valid queued primitive → CONTINUE for zero model calls.
Otherwise → PLAN, if its calls fit the remaining budget.
Three consecutive recoveries without progress → STOP.
Re-check every primitive. Consume each pending failure only once.

02 · ONLINE AGENT ROUND

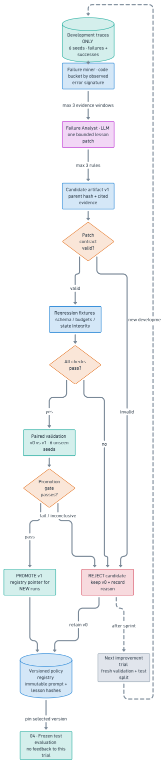
Independent proposals; dependent review; validated execution.



Agent contract
Planner and Explorer share frozen facts but do not see each other's proposal.
One failed proposer → review the survivor and mark degraded.
No survivors / failed Reviewer → recovery; never execute an unreviewed plan.
36 model attempts / 128 action attempts / 6 min per episode.
Retries consume extra slots; the held review slot cannot be stolen.

03 · OFFLINE IMPROVEMENT LOOP

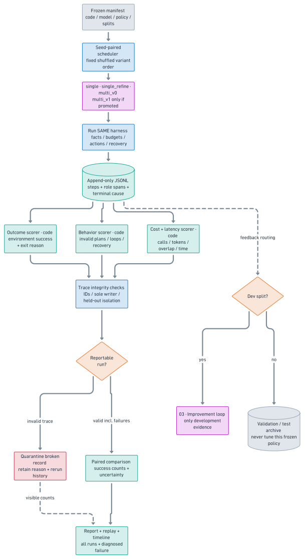
One candidate this sprint. No live self-modifying code.



Promotion gate · all required
0 integrity violations; all fixtures pass; 6 matched validation pairs complete.
No previously successful validation seed becomes a failure.
Gain ≥1 success OR equal success with ≥15% fewer model calls on common successes.
Mean calls ≤110% of parent; mean wall time and reported tokens each ≤125%.
Need ≥3 common successes for the efficiency route.
Fail / incomplete / missing token usage → keep v0. Small-sample engineering gate, not statistical proof.

04 · EVALUATION HARNESS

Outcome, behavior and cost are scored from actual events.


Frozen manifest
code / model / policy / splits
Seed-paired scheduler
fixed shuffled variant order
single · single_refine · multi_v0
multi_v1 only if promoted
Run SAME harness
facts / budgets / actions / recovery
Append-only JSONL
steps + role spans + terminal cause
Outcome scorer · code
environment success + exit reason
Behavior scorer · code
invalid plans / loops / recovery
Cost + latency scorer · code
calls / tokens / overlap / time
feedback routing
Trace integrity checks
IDs / sole writer / held-out isolation
Dev split?
yes
no
Reportable run?
invalid trace
valid incl. failures
03 · Improvement loop
only development evidence
Validation / test archive
never tune this frozen policy
Quarantine broken record
retain reason + rerun history
Paired comparison
success counts + uncertainty
visible counts
Report + replay + timeline
all runs + diagnosed failure

Evaluation contract
Development: seeds 0–5. Validation: 100–105. Test: 1000–1009.
Freeze model, prompts, lesson hash, budgets and order before test.
Compare single, single_refine, multi_v0; add multi_v1 only if promoted.
Score environment success, failure reasons, invalid plans, calls, tokens,
role selection, recovery and measured request overlap.
Infrastructure failures stay visible. No test feedback enters this trial.
Design specification; no gameplay results have been measured yet.